

zkRPC: Trustless Bitcoin data availability network

Kundu Chen, Jie Luo *

School of Computer Science and Engineering, Beihang University (BUAA), Beijing 100191, PR China
State Key Laboratory of Complex & Critical Software Environment (SKLCCSE), Beijing 100191, PR China

ARTICLE INFO

Keywords:

Blockchain
Bitcoin
Data availability
Zero-knowledge proof

ABSTRACT

The escalating size of data in the Bitcoin blockchain poses significant challenges in terms of block verification and storage efficiency, characterized as data availability issues. zkRPC is developed to address these challenges by leveraging a decentralized architecture and zero-knowledge proofs (ZKPs) to ensure data integrity and availability while minimizing the requirement of storage. The implementation of zkRPC involves distributing the blockchain data among nodes, where each node stores only a portion of the data but can verify the entirety using ZKPs. Theoretical analyses and practical evaluations of zkRPC demonstrate its effectiveness in reducing storage requirements while maintaining fast and reliable data verification across the network. Our approach not only enhances the scalability of Bitcoin but also improves the storage performance. Overall, zkRPC is shown as a suitable solution for trustless Bitcoin data availability.

1. Introduction

Bitcoin is becoming increasingly large—an unfortunate reality. Currently, a full Bitcoin node occupies 567.70 GB of space [1], making it cumbersome to operate on personal computers. Fortunately, the *simplified payment verification* (SPV) client allows users to verify transactions without needing the full blockchain. By downloading only block headers, which are significantly smaller than entire blocks, SPV clients efficiently confirm that a transaction is included in a block and thus part of the blockchain with a valid Merkle path [2]. The SPV is beneficial for ordinary users with limited resources, as it provides a balance between security and resource efficiency, enabling broader accessibility to Bitcoin's network.

However, this situation is gradually changing. With the introduction of the Taproot upgrade, Bitcoin scripting capabilities have been enhanced, giving rise to various of special techniques, which include:

- Assets that build upon Bitcoin, such as inscriptions, BRC20 tokens, and ordinal protocols [3].
- Sovereign rollup networks lock funds in Bitcoin.
- The client-side validation (CSV) program store their commitment data in the transactions.

These new techniques shares a common characteristic, which required the full transactions data in Bitcoin to ascertain their legitimacy, rather than solely through block header verification, as shown in Fig. 1. Regard these techniques are trending in the Bitcoin ecosystem, they have leads to several problems:

- The user must maintain the latest data from the Bitcoin network rather than only the block header [4], where the SPV client failed to meet the critic.
- The size of the SPV client is still too large for several scenarios, including mobile wallet/payment application, Bitcoin assets manager/marketplace in browser.
- The network situation of SPV client is not always ideal, as it crosses multiple countries/regions, which affect the experience of user.

Overall, the new features of Bitcoin led to the requirement for substantial storage capacity and computational hardware to process the data, rendering *data availability* (DA) a burgeoning problem in the Bitcoin network [5].

1.1. The data availability issue

In blockchain networks, the concept of data availability can be roughly defined as follows: while users can acquire specific data d , there must be corresponding proof p that demonstrates its correctness. The most common form of data availability is the Bitcoin full nodes, where any block can be verified along with its contained original transactions; similarly, the entire chain can be validated through all included blocks, which is shown in Fig. 2:

However, due to throughput and efficiency demands, user is not prefer to store all transactions and blocks. Even the SPV clients must rely on extra information (Merkle proof) offered by full nodes to

* Corresponding author at: SKLCCSE, School of Computer Science and Engineering, Beihang University (BUAA), Beijing 100191, PR China.
E-mail address: luojie@nlsde.buaa.edu.cn (J. Luo).

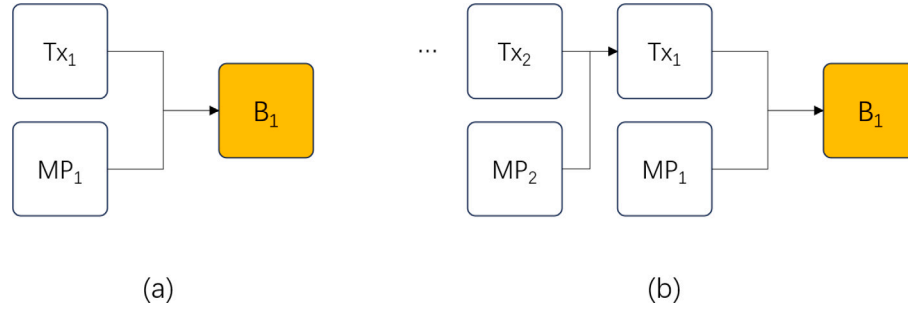


Fig. 1. The new challenge in Bitcoin and the limitation of SPV client. The white section represent the input, and yellow section represent the data stored by SPV client. (a) To verify a normal Bitcoin transaction, it only requires transaction Tx and its Merkle proof MP (for simplicity, we hide all the block headers). (b) To verify a BRC20 transaction, it requires the data of all related Tx_n , which is beyond the capability of SPV client. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

verify the transactions, failing to meet data availability requirements independently [6]. In ideal scenario, the user do not need to maintain any data locally. Therefore, the most common way to deal with data availability problem is leverage an *RPC* (Remote Procedure Call) provider, or *Indexer*, who offers API for users to query the on-chain data, like *Blockstream*, *Alchemy* [7]. Leveraging the data availability from the provider, the user can access the on-chain data without the block header, which is shown in Fig. 3:

Basically, the user can obtain on-chain data by just sending an HTTPs request. It greatly simplified the requirement of interacting with the Bitcoin network. However, the reliability of the data cannot be ensured in this circumstance [8]. The provider is capable of cheating the users with invalid blocks/transactions, as long as the users do not own the full blockchain to validate the data. The user have to trust the provider will behave honestly, which is not a *trustless* data availability solution. In response to this problem, two emerging solutions are currently employed:

Data Availability Committee (DAC): The DAC consists of a selected group of nodes responsible for verifying and ensuring the availability of data [9,10]. These committee nodes receive and store complete transaction data and jointly verify its integrity and correctness through various mechanisms, such as signatures or consensus protocols. Any blockchain participant can request data from the committee and obtain verification proof. This approach is designed to alleviate the burden on individual nodes while maintaining the network's security.

However, while the committee is composed of multiple nodes, its structure can lead to centralization of power, as these nodes play a crucial role in the validation and storage of data. If there is collusion among committee members or if they are targeted by malicious attacks, the entire system's trust and security could be compromised.

Data Availability Sampling (DAS): The DAS is a technique that allows nodes to download and verify only a part of a block to ensure the availability of the entire block's data. Nodes randomly select a small portion of data within a block for downloading and verification [11]. Based on statistical principles, the random sampling by many nodes significantly reduces the probability of data being maliciously hidden. If a node discovers data unavailability during its sampling, it issues an alert to inform other nodes in the network. This method enables even nodes with limited storage capabilities to verify block data availability without the need to download the entire block, facilitating decentralized services [12].

Although DAS reduces the risk of data being hidden by statistical methods, it cannot eliminate it. In extreme cases, this method could fail if a sufficient number of nodes are manipulated or fail to report unavailability. Additionally, if there is a large number of malicious nodes in the network or if the node distribution is highly uneven, the sampling results could be skewed, thereby affecting the accuracy of the verification.

Overall, the current data availability solution has limitations when deployed in the Bitcoin, the summarize of current solution is shown in the Table 1:

1.2. Our approach

In response to the challenges of centralization and correctness in data availability, this paper recognizes two essential aspects:

Zero-Knowledge Proof (ZKP): The ZKPs allow one party (the prover) to prove the correctness of a statement to another party (the verifier) without revealing any information other than the correctness of the statement itself. The essence of this technology lies in its ability to verify the correctness of data without sharing the actual data, thus ensuring privacy protection [13]. Although originally intended for privacy, ZKPs can address many issues in data availability: (1) ZKPs ensure that data can still be verified without the nodes accessing the complete data, which is crucial for the effectiveness of data availability solution; (2) Another key advantage of ZKPs is that the proofs generated are typically small in size and can be verified quickly, making them ideal for use in environments with limited storage space or computational resources [14].

The Proof-of-Work (PoW): The PoW is a classic mechanism employed in the consensus of Bitcoin, characterized by its use of computational power to ensure block security [15]. Nodes must solve a cryptographic hash puzzle to legitimize their blocks, with the proof of this work stored in the block header. The robust network and computational power make the blocks in Bitcoin easy to verify yet difficult to forge, and to date, there has been no successful computational attack on the Bitcoin network [16]. From the perspective of Zero-Knowledge Proofs, as long as a series of blocks meet the PoW requirements, their correctness and finality can be simultaneously confirmed. In contrast, in other consensus mechanisms such as Proof-of-Stake (PoS), although ZKP can prove correctness, as creating forks in the PoS chain is too easy, the finality of blockchain cannot be assured [17]. In such cases, even with ZKP, data availability cannot always be guaranteed, thus highlighting the crucial importance of the PoW mechanism.

Based on these insights, this study propose *zkRPC*, a decentralized network to provide a trustless data availability service. *zkRPC* utilizes zero-knowledge proofs of Proof-of-Work to generate compact proofs that can effectively verify the integrity and correctness of transactions and block data without revealing the raw data itself. These proofs are small-sized, and easily verifiable, which significantly reduces the demand for network bandwidth and storage. The outline of *zkRPC* is shown in Fig. 4:

zkRPC allows participants with limited resources to efficiently engage in the network's verification process without handling complex computation processes or storing large amounts of data. Specifically:

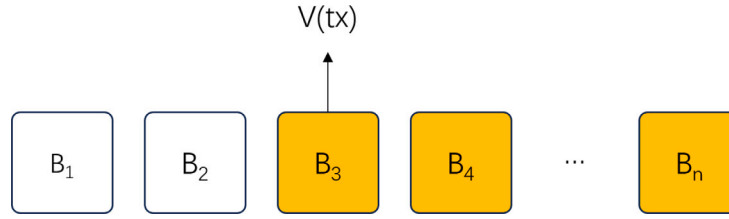


Fig. 2. The standard way to verify a transaction tx in B_3 , is to validate the entire historical chain (yellow section). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

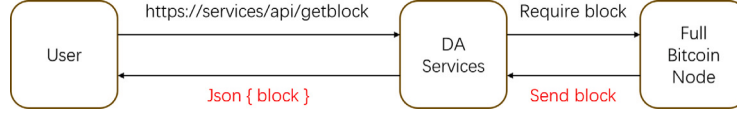


Fig. 3. The workflow of RPC provider and the data availability issue in Bitcoin. The RPC provider (Bitcoin fullnode) offer API to user to query on-chain data. The user cannot verify the data, therefore the provider is capable of sending fake blocks (red). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 1
Existing Data Availability Solution.

Name	Advantage	Shortcoming
Full Node	Capable of validate any blockchain data	Require large space
RPC (Indexer)	Require no space, fast response	Cannot validate the responded data
DAC	More trusted than RPC provider	Centralized, committees can perform collective evil
DAS	Decentralized data validation	Require extra space, not fully reliable

- **Efficient Data Management:** Through the separation and selective distribution of data by supervisor nodes, zkRPC allows network nodes to hold only parts of the complete blockchain state, while the user can completely store no state at all. This approach significantly decreases the storage requirements for individual nodes, facilitating scalability and faster data processing.
- **Verifiability with Zero-Knowledge Proofs:** The essence of zkRPC's functionality lies in its use of *ZKPs* to verify the integrity and completeness of the Bitcoin data without disclosing the data itself. Each node uses these proofs to ensure the correctness of transactions, thus maintaining privacy and security while ensuring data availability.
- **Decentralized Data Availability Network:** zkRPC dynamically handles data requests, as the number of nodes increases, the data storage requirement per node decreases, further supporting the network's decentralization.

The rest of the paper is arranged as follows: Section 2 provide a comprehensive exposition of ZKP, Bitcoin, and data availability. Section 3 dive into the details of the zkRPC, demonstrating its architecture, workflow, and essential module. Section 4 dedicated an in-depth analysis of zkRPC to show its capability of DA. Section 5 conduct experiments to observe the performance of the zkRPC under various scenarios and prove its efficiency. Finally, Section 6 discusses the implementation of zkRPC, Section 7 introduces the referenced work, and Section 8 presents the conclusions and discusses the implications of our research.

2. Preliminary

2.1. Background on zero-knowledge proofs

Zero-knowledge proof (ZKP) is a scheme that allows a prover to construct a statement and its corresponding proof, and submit this to a verifier for validation without revealing any information. More specifically, the information known only to the prover define as i_s , the common information shared between the parties as i_p , the submitted proof as P , and the statement to be verified is S . S can be regard as the constraint of the ZKP. A zero-knowledge proof must satisfy several properties:

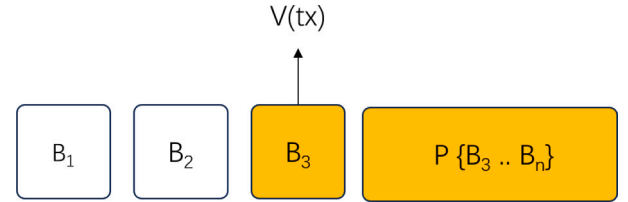


Fig. 4. The outline of zkRPC. To verify a transaction tx in B_3 , the verifier only needs block B_3 and the zero-knowledge proof-of-work of $B_4 \dots B_n$ (yellow section). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

- **Correctness:** By validating P , the verifier can ascertain that the prover possesses i_s .
- **Soundness:** Provided that the prover holds i_s , the constructed P will always pass the verifier's scrutiny.
- **Zero-knowledge:** The verifier cannot derive i_s from the proof process.

According to the description above, the process of ZKP can be define as follows 1:

Definition 1. The ZKP prover of $\langle i_s, i_p \rangle$ is a function $\mathcal{ZP}$ where $\mathcal{ZP}: \{ i_s, i_p, S \} \rightarrow P$, where $\langle i_s, i_p \rangle \models S$ indicate i_s, i_p should satisfied the statement S (1):

$$\mathcal{ZP}(i_s, i_p, S) = P, \text{ if } \langle i_s, i_p \rangle \models S; \quad (1)$$

Meanwhile, the verifier $\mathcal{ZV}$ is a function $\mathcal{ZV}: \{ i_p, S, P \} \rightarrow res \in \{1, 0\}$, where 1 is valid, and 0 is invalid (2):

$$\mathcal{ZV}(i_p, S, P) = \begin{cases} 1, & \text{if } P = \mathcal{ZP}(i_s, i_p, S); \\ 0, & \text{otherwise;} \end{cases} \quad (2)$$

In practice, the statement S can be implementation by a special kind of schema named *Circuit*. The $\mathcal{ZP}$ and $\mathcal{ZV}$ can be regard as practical programs that satisfies the properties discussed above, which is shown

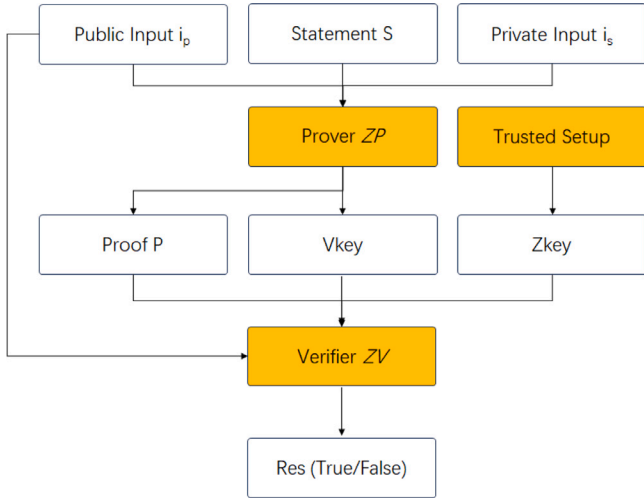


Fig. 5. The framework of the ZKP prover and verifier. The white section represent data, and the yellow section represent program. (1) Through prover ZP to get a proof to prove i_s and i_p satisfied S ; (2) From statement S generate RICS and verification key $Vkey$, meanwhile $Zkey$ is a public parameter setting by *trusted setup*. (3) Through verifier ZV to validate if P match the $Vkey$ and i_p . (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

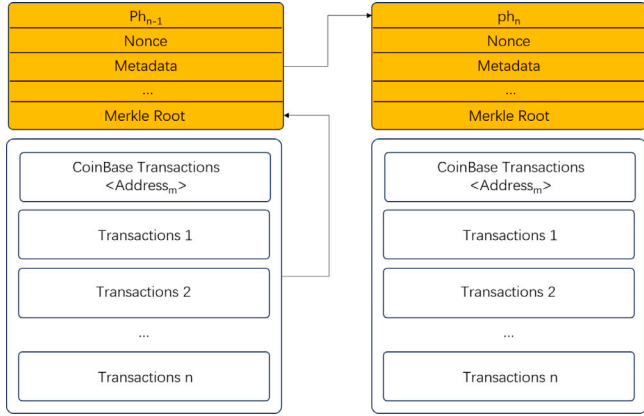


Fig. 6. The construction of Bitcoin block. (1) The *Merkle Root* is the digest of the block body (white section); (2) The hash value of the previous block header is attached to the block header (yellow section) of the current block. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

in Fig. 5:

The major shortcoming of ZKPs is the proving/verifying cost. When the complexity of statements grow, the time consume will become significant [18]. Extensive and excellent work has been done on the implementation and optimization of zero-knowledge proofs, which is detailed in various sources [19].

This work use *Circom*, a ZKP circuit language based on *Groth16* and *Plonk* schema [20] to build ZP and ZV . Furthermore, for the *Circom* framework, the statement S in ZP is implemented by Rank-1 Constraint System RICS, and S in ZV is implemented by verification key $Vkey$.

2.2. Bitcoin overview

Bitcoin has several of essential components, which are transactions, blocks, and the data verification process, which have been explicitly explained in existing work [21]. An overview of the Bitcoin block is

shown in Fig. 6:

Transactions: In the Bitcoin network, the atomic data is a transaction, denoted as tx_n^b , where the subscript indicates its index, and the superscript indicates its affiliation to block b . A simplified version of transaction is denote in (3), where $\mathcal{M}$ is the metadata of tx :

$$tx_n^b = \{tx_{in}, tx_{out}, \mathcal{M}, sig_{tx}\} \quad (3)$$

From (3), a valid transaction should be satisfied: (1) tx_{in} and tx_{out} should be valid, as the payment in bitcoin relies on the unspent transaction; (2) the signature of tx should contain the entire remaining elements $tx_{in}, tx_{out}, metadata$, otherwise it should be invalid. The verification function of tx is denote as V_{tx} , it can be express as (4):

$$V_{tx}(tx) = \begin{cases} 1, & \text{if } V_{sig}(sig_{tx}) \wedge V_{tx}(tx_{in}) \wedge V_{tx}(tx_{out}); \\ 0, & \text{otherwise;} \end{cases} \quad (4)$$

Where V_{sig} is (5):

$$V_{sig}(sig_{tx}) = \begin{cases} 1, & \text{if } sig_{tx} = sig(tx_{in}, tx_{out}, \mathcal{M}); \\ 0, & \text{otherwise;} \end{cases} \quad (5)$$

When any block is broadcast to the Bitcoin network, receiving miners verify whether the hash value of its block header bh is less than the target value t specified by the protocol, that is the PoW mechanism. Due to the irreversibility of hash functions, to make the hash value of bh meet the requirement, the miners must continuously change the value of *nonce* to alter $hash(bh)$ to obtain a valid block header, and perform a vast number of hash calculations. The block header verification function V_{bh} can be expressed in Eq. (6):

$$V_{bh}(bh) = \begin{cases} 1, & t > hash(bh); \\ 0, & t \leq hash(bh); \end{cases} \quad (6)$$

Currently, Bitcoin employ *SHA256* as hash algorithm, and use *secp256k1* to encrypt the address, which is different from mainstream *Ed25519*, and does not meet the standard of post quantum crypto currently [22,23].

Block: Blocks are the basic units that constitute the blockchain, denoted as $b_n = \langle bh_n, bb_n \rangle$, where the subscript indicates its height. Each block consists of the block body bb_n , which contains a collection of transactions, i.e., $bb_n = \{tx_1 \dots tx_m\}$, and the block header bh_n . The structure of the block header is as shown in (7):

$$bh_n = \{root(bb_n), ph, nonce, \mathcal{M}\} \quad (7)$$

In (7), the $root(bb_n)$ is the Merkle tree root of all the transactions inside bb_n . When any block is broadcast to the blockchain network, receiving miners verify (1) whether the preimage of $root$ matches bb_n ; (2) whether ph matches the hash value of the block header of bh_{n-1} ; (3) whether the hash value of bh_{n-1} is less than the target value t specified by the protocol. for simplicity, the t is set as a constant; (4) all the transactions in the block are valid. The verification function of b is denote as V_b , it can be express as (8):

$$V_b(b) = \begin{cases} 1, & \text{if } \forall tx_n \in bb, V_{tx}(tx_n) = 1 \wedge \\ & root(bb_n) = R(bb_n) \wedge \\ & ph_{bh} = hash(bh_{n-1}) \wedge \\ & V_{bh}(bh_n) = 1; \\ 0, & \text{otherwise;} \end{cases} \quad (8)$$

Where R is the function that calculates the Merkle root, and there are existing works that explicitly introduce this algorithm [24].

Bitcoin leverages the Merkle root to check whether a transaction tx_n is inside a block. The hash of a transaction is denote as H_{tx_n} and $hash(tx_n)$. Observing bh , it contains the summary of the block body bb , which is the Merkle Root $root(bb)$. Specifically, the structure of the

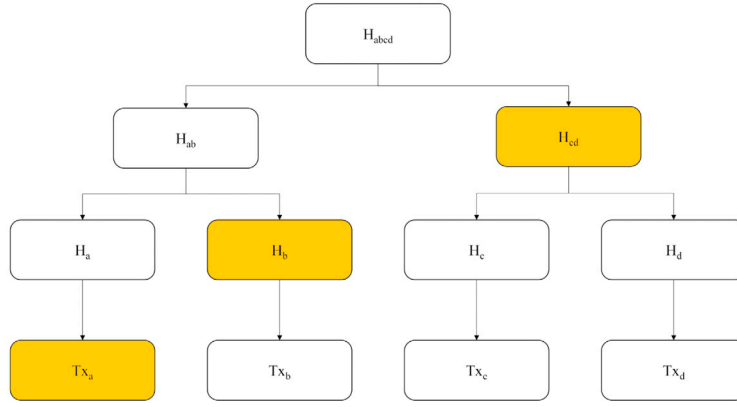


Fig. 7. Illustration of a Merkle tree in Bitcoin, the yellow section shows a Merkle path. When all yellow section gathered, a transaction can be verified. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Merkle tree is illustrate in Fig. 7:

In the figure, H represents a hash value, and the subscript of H indicates which transactions constitute the pre-image of this hash, for example, H_{ab} represents the hash value formed by transactions tx_a and tx_b , tx denotes a transaction. It is observable that the Merkle root H_{abcd} encompasses all the transactions. Given the structure of $root(bb)$, it is found to facilitate rapid verification of whether a transaction is contained within bb . To verify a transaction, the transaction tx_n , along with all hashes H containing tx_n , known collectively as the *Merkle path*, denoted here as $mp = \{H_1, H_2 \dots H_n\}$, are required. As illustrated above, to find if transaction tx_a exists in the tree, one needs to input tx_a , H_b , H_{cd} , and compute $hash(hash(hash(tx_a), H_b), H_{cd})$ to compare it with H_{abcd} . If they match, then tx_a is confirmed to be in *Root*; otherwise, it is not. It can be described as follows (9)(10):

$$inc(h, mp, i) = \begin{cases} hash(h, H_i) & \text{if } i = 1 \\ hash(inc(i-1), H_i) & \text{if } i > 1 \end{cases} \quad (9)$$

$$V_m(tx, mp, root) = \begin{cases} 1, & \text{if } \sum_{i=1}^n inc(hash(tx), mp, i) = root \\ 0, & \text{otherwise;} \end{cases} \quad (10)$$

In this example, the Merkle path for transaction tx_a consists of H_b and H_{cd} .

2.3. Data availability

According to the previous description, data availability in the context of Bitcoin can be formally defined, and it can be expressed as 2:

Definition 2. Data availability is defined as a property that ensures that all data necessary to fully validate the blockchain is accessible to all network participants, and can be examined by function D shown in 2:

$$D(b) = \begin{cases} 1, & \text{if } b \in C \wedge V_b(b) = 1 \\ 0, & \text{otherwise;} \end{cases} \quad (11)$$

where: $D(b)$ is the data availability of block b , C represents the entire chain, which contains all blocks or transactions, and b is the request block. From the 2, satisfying DA involves two critical factors:

- **Completeness:** Data availability requires access to all transactions within the entire chain C . This means that every transaction that is part of the blockchain must be retrievable and accessible to ensure the ledger's integrity.
- **Verifiability:** Any block provided must be verifiable by a verification process denoted as V_b . This ensures that the block is not only present but also valid according to the blockchain's consensus rules.

These factors are essential to maintain and build the trust and functionality of the DA network.

3. zkRPC breakdown

3.1. Overview

Currently, data availability solutions in Bitcoin face two main challenges: (1) the majority of nodes in the network do not possess (complete) Bitcoin data, which compromises the *verifiability* of the data; and (2) to ensure *completeness*, nodes must store all Bitcoin data, consuming significant storage space.

To address the issue of verifiability, zkRPC response the user with a tuple $d = \langle b, P \rangle$, where P is constructed through a zero-knowledge proof. With P , user is capable of verify the correctness of b , this work identity this capability as Lemma 1, and the prove is shown as follow:

Lemma 1. The function of *proven a block b is valid*, can be converted into a zero-knowledge proof P by a prover $\mathcal{ZP}$, and be verified by a verifier $\mathcal{ZV}$.

Proof. By Definition 1 and Eq. (1), P can be obtain by $\mathcal{ZP}: \{i_s, i_p, S\} \rightarrow P$. The entire blockchain can be denoted as $C = \{b_1, b_2 \dots b_n\}$, from Definition 2, the $\mathcal{ZP}$ can be converted into:

$$\mathcal{ZP} : \{C, b, D(b)\} \rightarrow P \quad (12)$$

Where C is the private input i_s , b is the public input i_p , $D(b)$ is the statement derive from 2. Through $\mathcal{ZP}$, the necessary parameter of $\mathcal{ZV}$ can be obtained, where:

$$\mathcal{ZV}(b, D(b), P) = \begin{cases} 1, & \text{if } P = \mathcal{ZP}(C, b, D(b)); \\ 0, & \text{otherwise;} \end{cases} \quad (13)$$

Therefore, the valid proof P of the Proof-of-Work verification function 2 can be generated to prove the block b is valid and construct a $\mathcal{ZV}$ to verify the proof P . $\square$

By treating C as a private input i_s , it absolve the necessity for users to possess full Bitcoin blockchain during the verification of b . This significantly reduces the amount of data that needs to be transmitted.

To tackle the problem of completeness, each node in zkRPC stores only a part of the Bitcoin data, while the rest is represented by zero-knowledge proofs. Specifically, let the node with index i be denoted as N_i , and its state can be expressed as (14):

$$N_i = \{P_i, C', P_r | C' \in C\} \quad (14)$$

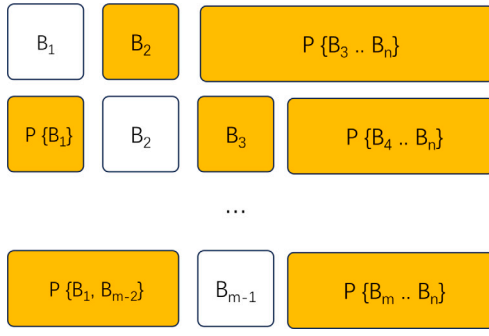


Fig. 8. The concept of zkRPC. Each node holds part of the full state (yellow section) while containing the proof. In practice, a node shall maintain multiple blocks depending on the total number of nodes in the network, it is quite like a sliding window. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

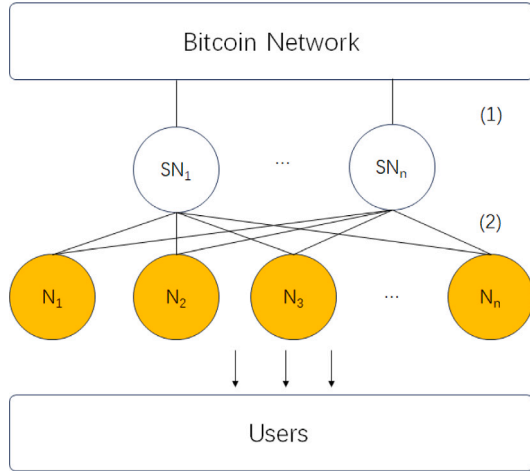


Fig. 9. The architecture of zkRPC. (1) The supervisor node SN received the full data from the Bitcoin network. (2) The validator node N connected with each other and SN to received the newest data and proof in P2P manner.

Using proofs to replace the actual data storage allows zkRPC's nodes to ensure the integrity of requests while only maintaining a subset of the entire blockchain data. Combined with the tuple, it can achieve trustless and reliable DA in the network, the concept of zkRPC is shown in Fig. 8:

Separating the full state of Bitcoin into multiple nodes further lowers the storage requirement of each node and achieves decentralization in the zkRPC network.

3.2. Architecture

Overall, zkRPC operates as a decentralized network. Communication among its nodes is based on a peer-to-peer (P2P) model, yet it also requires connectivity to the Bitcoin network to access up-to-date data. The general structure of the network is illustrated in Fig. 9:

In zkRPC, there are two distinct types of nodes: the supervisor node SN and validator nodes N . As depicted, the SN must operate a full Bitcoin node to request complete data structures from the Bitcoin network. This necessity arises because accurate and complete Bitcoin data can only be obtained by running a full node (e.g. using RPC-JSON API is considered unreliable). The difference between SN and N is the hardware requirement, where in the initial stage of zkRPC, the storage and bandwidth load of SN can be heavier than N . This work denoted the entire zkRPC network as $\mathcal{N} = \{SN, N\}$

After obtaining the complete chain C from the Bitcoin network, the SN then partitions this network data and distributes it among the connected N nodes. All ordinary nodes connected to the supervisor SN_n are denoted as (15):

$$SN_n = \{C, N_1, \dots, N_n\} \quad (15)$$

Where the current state of each node being $N_n = \{C_n\}$, it ensures that $\forall N_i \in SN, N_0 \cup N_1 \dots \cup N_i = C$. This distribution allows ordinary nodes in the network to spread their data load, reducing the storage and request pressure on individual nodes. Overall, the workflow of zkRPC can be described as follows:

- **Initialize:** SN connected with Bitcoin network, and as many validator nodes N as possible.
- **Separate:** SN divide the complete state C to C' and generate the corresponding ZKP for the connected N .
- **Distribute:** SN sends the ZKP and separated state C' to N .
- **Validate:** N validates the ZKP and states C' locally, if valid, the validator will be active and receive data requests from users.
- **Update:** SN continuously monitors the Bitcoin network and updates the latest block.

The workflow of zkRPC illustrates the network's ability to handle requests dynamically, relying on the supervisor node SN to facilitate the retrieval and delivery of data across the network. Such a mechanism ensures that even if individual nodes do not hold all data locally, the system as a whole can satisfy data requests efficiently and maintain high levels of availability and reliability.

3.3. Circuit of zero-knowledge PoW

For zkRPC, a critical step is constructing the Zero-Knowledge Proof (ZKP) corresponding to the Proof-of-Work (PoW). Specifically, this paper achieves this by implementing the corresponding prover and verifier, as previously discussed in 5. The key to implementing ZP and *mathcal{ZV}* lies in transforming the block verification function $V_b(b)$ into a ZKP statement and then realizing it through a circuit. Observations from equations indicate that it is unnecessary to use ZKP to verify each transaction within a block when the complete block information is available. Having the original transaction data allows users to easily construct a Merkle tree and verify its correctness against the block header. Therefore, this paper focuses solely on implementing the PoW process between blocks.

This work start from the circuit of a single block: the hash value of previous block header $hash(b_{n-1})$ should be included in b_n , while the hash value of current block $hash(b_n)$ should be less than the target t . the circuit can be demonstrated in pseudocode as shown in algorithm 1:

Algorithm 1: Circuit of Block Verification

Input: Previous block hash $pHash$, block headers bh , Target number T

Output: Boolean indicating if block is valid

```

1 Function CheckBlock( $bh, pHash$ ):
2    $cHash \leftarrow SHA256(bh)$ 
3   if  $getpHash(bh) = pHash$  then
4     return false
5   end
6   if  $T > cHash$  then
7     return false
8   end
9   return true

```

The *Circuit of Block Verification* is integral to verifying the integrity of blockchain blocks within the zkRPC framework. It primarily focuses on validating the cryptographic hash of each block against predetermined standards. The algorithm processes each block header, which

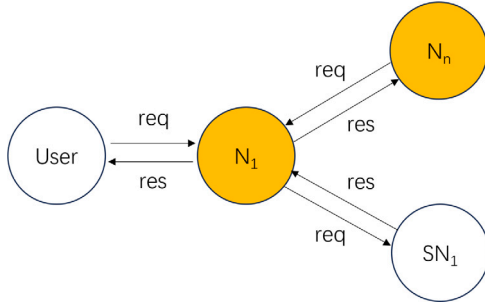


Fig. 10. The zkRPC footstep of data request. (1) User send *req* to node N_1 in the zkRPC network. (2) If N_1 contained the request data, it return *res*. (3) If not, N_1 will try to reach other validator node N_n for *res*. (4) If none of the validators is capable of building *res*, N_1 will send the request to the supervisor SN_1 for *res*.

contains 80 bits in the Bitcoin standard. The result of the function $getpHash(bits, pHash)$ should match the *pHash*, while the target should be greater than the *cHash*.

Through circuit 1, zkRPC can sequentially pass block data and build the circuit for a blockchain, verifying each block against its PoW criteria. The circuit is shown in the algorithm 2:

Algorithm 2: Circuit of Sequential Block Validation

Input: Chain C , Number of blocks $BLOCK_COUNT$, block header bh

Output: Boolean indicating if block is valid

```

1 Function Main( $BLOCK\_COUNT, bh, C$ ):
2   for  $i \leftarrow 0$  to  $BLOCK\_COUNT - 1$  do
3      $pHash \leftarrow getpHash(bh)$ 
4      $S \leftarrow CheckBlock(bh, pHash)$ 
5     if  $S$  then
6        $bh \leftarrow C[bh.height - i]$ 
7       Continue
8     else
9       return false
10    end
11  end
12  return true

```

The *Circuit of Sequential Block Validation* orchestrates the verification process for a series of blockchain blocks. It utilizes circuit 1 to validate each block sequentially based on the Proof of Work consensus mechanism. The algorithm initializes with the previous block's hash and iteratively processes up to $BLOCK_COUNT$ blocks. Each block's integrity is verified by comparing its hash against the computed hash from the previous block's data, ensuring continuity and linkage in the blockchain. The final output is the hash of the last block, confirming the end of the validation sequence. This structure is crucial for maintaining the blockchain's integrity across multiple blocks, enhancing security and reliability.

The circuit implementation ensures that each block processed by zkRPC meets the rigorous requirements of the Proof-of-Work, confirming both its correctness and finality without necessitating a ZKP for each transaction within the block.

3.4. Block request

To provides data availability solution, zkRPC shall handle the block request from user. Typically, a user initiates a data request to any validator node N within the zkRPC network. The footstep of the data request is shown in Fig. 10:

To ensure the functionality of nodes, they shall be alive and accessible. The aliveness of the nodes and networks is defined as follows 3:

Definition 3. Denoted $N \models \mathcal{A}$ as a node N_i is alive. When there exists a subset of SN and N where each nodes is alive, the network $\mathcal{N} \models \mathcal{A}$

In an optimistic scenario, if the requested block b is contained within N_1 , the node will directly return the corresponding result tuple. In a pessimistic scenario, where $b \notin N_1$, the N_1 attempts to retrieve the data by communicating with other validator nodes N_n within the network. If $b \notin N_n \vee N_n \not\models \mathcal{A}$, then the N_1 should obtain b from the supervisor node SN .

Next, this work examine the details of the data request. When a user makes a request, the node N sends a request data tuple $d_{req} = \langle b, P \rangle$, where P is constructed via $\mathcal{ZP}(i_p, b, S)$. Upon receiving d , the user can verify the results using $\mathcal{ZV}(b, S, P)$. It is known that i_p and P are predefined for $\mathcal{ZV}$ and $\mathcal{ZP}$. The crux of constructing a ZKP lies in forming the statement S required to verify b , and constructing the necessary i_s . For this paper, the statement can essentially be derived from the block's verification function $V_b(b)$.

Let us consider the properties that the statement aims to describe. Initially, the statement must contain $V_b(b)$, i.e., $V_b(b) \subset S$. Observing conditions (1), (2), and (4) from Eq. (8), it is clear that these can be independently verified with complete data of b ; however, for condition (3), i.e., $ph_{bh_n} = hash(bh_{n-1})$, involves recursive verification through the entire C due to the requirements of completeness. This recursive requirement suggests that $V_b(b_{n-1}) = (b_{n-1}) = 1$, and so forth for b_{n-1} , which must satisfy $ph_{bh_{n-1}} = hash(bh_{n-2})$. Ultimately, the statement S demands verifiability of all blocks preceding b in C , i.e., $\forall b_i \in C, V_b(b_i) \subset S$. Thus, $i_p = C$ for $\mathcal{ZP}$.

After examined the S and i_p , this statement can be implemented using a circuit. For simplicity, this work will not detail the construction of the Circom circuit but will describe the circuit program using the algorithm below, termed as the *Request Statement*, denoted S_r . The construction is shown in algorithm 3:

Algorithm 3: Request Statement S_r

Input: Chain C , Block b_n

Output: Statement S_r

```

1 Get latest height  $h \leftarrow |C|$ ;
2 Get subset  $C' \leftarrow C \setminus \{b_h, b_{h-1} \dots b_n\}$ ;
3  $ph_{bh_i} \leftarrow hash(bh_{i-1})$ ;
4  $v \leftarrow ph_{bh_i}$ ;
5 if  $v$  is False then
6   | Break and output error;
7 end
8 for each  $b_i$  in  $C'$  do
9    $v \leftarrow V_b(b_i)$ ;
10  if  $v$  is False then
11    | Break and output error;
12  end
13   $S_r \leftarrow S_r \wedge v$ ;
14 end
15 Return  $S_r$ ;

```

From the preceding formulation, when constructing S , it is unnecessary to verify the conditions of b_n in $V_b(b)$ other than condition (3) as part of the zero-knowledge proof. This is because once the user receives the complete block, they are capable of verifying these conditions locally. However, for the historical block, all of the verification need to be conducted through ZKP. As a result, denote $N_i = C_i | C_i \in C$, the block request process $req(b)$ can be expressed in (16):

$$req(b) = \begin{cases} \langle b, P \rangle, & \mathcal{N} \models \mathcal{A} \wedge \mathcal{ZP} = 1; \\ 0, & \text{otherwise}; \end{cases} \quad (16)$$

This methodology allows the user to confirm whether the received data d satisfies both completeness and verifiability. By delegating local verifications of conditions not reliant on the entire chain's context and utilizing ZKP for those that do, zkRPC efficiently balances between minimizing the computational overhead and maintaining the integrity and security of the data verification process.

3.5. Transaction request

This work next considered an alternate situation: the user requests only a single transaction data tx , which is indeed a common API service. To add this into the zkRPC, it should first examine the verification process of transaction tx , according to Eq. (4), (5), (8), it can be separated into several steps:

- **Step 1:** Verify the transaction tx is valid by checking the signature and transaction tx is inside the block b .
- **Step 2:** Verify the block b is valid by checking the difficulty of PoW.
- **Step 3:** Verify the whole blockchain is valid by checking each block's previous block hash and PoW.

Steps 2 and 3 are actually included in the circuit 3. However, it required a full block to verify, which size is approximately 1 Mb, which is overwhelmingly larger than a single transaction. To address this issue, this work discovered that during the algorithm 3, only block header bh is necessary to generate a valid proof P . Therefore, Step 1 can be verified by simply checking the block header bh and the Merkle path by Eq. (10). It can be expressed as (17):

$$req(tx) = \begin{cases} \langle bh, P, tx, mp \rangle, & \mathcal{N} \models \mathcal{A} \wedge V_m = 1 \wedge \mathcal{ZP} = 1 \\ 0, & otherwise; \end{cases} \quad (17)$$

Therefore, when the user requires transaction data, the nodes in zkRPC shall only deliver the proof P , block header bh , transaction tx , and Merkle path mp , which is obviously much smaller than sending an entire block.

3.6. Data storage

For the supervisor node SN , which is responsible for obtaining the initial data from the Bitcoin network, it is imperative to store all data. Therefore, the key to reducing the network's storage load lies in minimizing the storage requirements of the validator nodes N . Revisit the workflow described in Section 3.2, the data storage process can be reorganized as follows:

- **Initialize:** SN retrieves the complete state C from Bitcoin and identifies all connectable N .
- **Separate:** Depending on the number of connected N , the complete state C is divided into $\{C_1, C_2, \dots, C_n\}$.
- **Distribute:** SN sends $d_{up} = \langle P_u, C_i \rangle$ to each N_i .
- **Validate:** N validates d_{up} . Upon receiving, if success, it store the d_{up} in the local storage $N = \{d_{up}\} = \{C_i, P_u\}$; If fails, the process reverts to *Distribute* where N requests d_{up} again from SN .
- **Update:** SN updates the complete Bitcoin state reassesses the number of nodes and reverts to *Initialize*.

A crucial aspect of this process is the construction of the correct proofs P_u , which represent the verifiability of the data before and after the segment C_i . In practice, this work merged them to $d_{up} = \langle P_u, C_i \rangle$. The construction of S_u , corresponding to P_u , follows a methodology similar to algorithm 3, as shown in algorithm 4:

The P_u ensures the verifiability of all remaining blocks, allowing the validator nodes N to maintain the integrity of the data availability solution while only storing a subset C_i of the Bitcoin data. This strategy effectively reduces the storage burden on individual nodes and preserves the completeness and verifiability of data across the network. Furthermore, it is evident that as the number of nodes N increases, the amount of data each node needs to store decreases. This reduction in per-node data requirement facilitates greater decentralization of the network.

Algorithm 4: Updated Statement S_u

Input: Complete chain state C , Segment C_i

Output: Updated Statement S_u

```

1 Get subset  $C' \leftarrow C \setminus C_i$ ;
2 for each  $b_i$  in  $C'$  do
3    $v \leftarrow V_b(b_i)$ ;
4   if  $v$  is False then
5     Break and output error;
6   end
7    $S_u \leftarrow S_u \wedge v$ ;
8 end
9 Return  $S_u$ ;
```

3.7. Summary

Through the framework of zkRPC, this work adeptly combine ZKP techniques with an innovative network architecture to address critical challenges in traditional blockchain frameworks.

- **P2P Network Architecture:** Utilizing a peer-to-peer model, zkRPC ensures that data distribution and verification tasks are decentralized. Each node operates independently and contributes to the network's robustness and fault tolerance.
- **Zero-Knowledge PoW:** zkRPC leverages ZKP to ensure that all transactions within the network are verifiable without necessitating the storage of the entire blockchain data by every node, and significantly reduces the storage requirements while enhancing privacy and security.

In summary, zkRPC addresses the limitations of data availability which lack decentralization and scalability.

4. Property analysis

4.1. Assumptions

Before the Analysis of zkRPC, several necessary assumptions should be declared, which include:

- **Immutability:** The Bitcoin network shall always be active, and ensures SN update the newest C . Meanwhile, the Bitcoin network provides righteous data and ensures $C = \{b_n \dots b_0\}$ is always valid, where $V(C) = 1$.
- **Reachability:** The latency in providing transactions, denoted as T , must be less than a predefined threshold Δ , where $0 < T < \Delta$.

The assumptions stated in this paper are made under *weak consistency assumption*, while simultaneously ensuring that the Bitcoin network remains secure. These assumptions are reasonable and also appear in numerous other prominent works [17,25].

4.2. Data availability

To demonstrate that zkRPC provides reliable Data Availability, it is essential to verify that it fulfills the three properties mentioned in **Definition 2: Completeness and Verifiability**. Here, this work focus on proving each property systematically.

Lemma 2 (Completeness). For any node N in a network $\mathcal{N} \models \mathcal{A}$, for all requested data b such that $b \in C$, N is capable of returning $d = \langle b, P \rangle$.

Proof. Set $b \in C$ and $N = \{C_i, P\} \in \mathcal{N}$ and the complete blockchain data is C , $C_i \in C$. The data b is either directly stored at N or verifiable through zero-knowledge proofs P that N can generate or retrieve. When $\mathcal{N} \models \mathcal{A}$:

- **Optimism:** If b is within N 's local subset $b \in C_i$, according to (16), N directly returns $d = \langle b, P \rangle$.
- **Pessimistic:** If b is not within N 's subset $b \notin C_i$, N utilizes the other nodes $N_i \in \mathcal{N}$ for $req(b)$. When $\mathcal{N} \models \mathcal{A}$, N_i is always reachable $N_i \models \mathcal{A}$. According to *separation* of C in data storage process, when $C = \{C_1, C_2 \dots C_i\}$ and $b \in C$ is satisfied, it can derive $\exists n, b \in C_n$. Therefore according to (16), node N_n shall returns $d = \langle b, P \rangle$.

Thus, as long as $b \in C$ and the network $\mathcal{N}$ functions correctly ($\mathcal{N} \models \mathcal{A}$), N will always be able to return $d = \langle b, P \rangle$, satisfying the property of completeness. $\square$

This proof highlights the effective distribution and verification mechanisms provided by zkRPC, ensuring that any data part of the blockchain C can be verified and returned by any node N in the network. Subsequently, this work will analyze and demonstrate the properties of Verifiability.

Lemma 3 (Verifiability). For any $b \in C$, there always be $d_{up} = \langle C_i, P_u \rangle$ where $\mathcal{ZV}(C_i, S, P) = 1$, and $d_{req} = \langle b, P \rangle$ where $\mathcal{ZV}(b, S, P) = 1$.

Proof. Let b be any block within the complete blockchain data $b \in C$ and $d_{req} = \langle b, P \rangle$ be the data and proof tuple updated or held by a node N within the zkRPC network, where P be the zero-knowledge proof associated with b . Additionally, $d_{up} = \langle C_i, P \rangle$ represents any request for C_i issued between nodes. To prove that $\mathcal{ZV} = 1$, consider the following:

- **Verify d_{req} :** To verify $\mathcal{ZV}(b, S, P) = 1$, it should first obtain P by $\mathcal{ZP}(C, b, S) = P$ from $N_i = P_u, C_i$, where b, S is public data, and C can be obtained from $\mathcal{ZP}(b, S, P) \cup P_u = P$. According to (2), the $\mathcal{ZV}(S, C, P) = 1$.
- **Verify d_{up} :** To verify $\mathcal{ZV}(C_i, S, P) = 1$, it need to obtain P_u by $\mathcal{ZP}(C, C_i, S) = P_u$ from supervisor SN , where C can be directly obtain from SN while $C \in SN$. According to (2), the $\mathcal{ZV}(C_i, S, P) = 1$.

Therefore, for any $b \in C$, given the correct construction and validation of the zero-knowledge proof P , $\mathcal{ZV}(S, C, P)$ will always return 1, thereby satisfying the condition of verifiability as stated in the lemma. $\square$

This proof relies on the correct implementation and functioning of the zero-knowledge proof mechanisms within zkRPC, as well as the network condition should always be reachable.

4.3. Decentralization

The architecture of zkRPC, which includes the presence of supervisor nodes SN , might initially suggest a potential for centralization. However, the system is designed to ensure decentralization through its operational dynamics and data distribution protocols. Here, this work analyze and prove the decentralized nature of zkRPC. This work first defined decentralization, which is quantified by the variance in the number of times nodes return information over infinite rounds. Specifically, the decentralization can be defined using the concept of variance, as shown in (18):

$$D_k = \frac{\sum_{i=1}^n (X_k^{N_i} - \bar{X})^2}{n-1} \quad (18)$$

Where $X_k^{N_i}$ is the number of times of a network node N_i returns $req(d)$ in k rounds, and n is the number of nodes. After defined the decentralization, this work demonstrated that zkRPC can satisfy the property 4:

Lemma 4 (Decentralization). When the number of SN equal to N , the decentralization D_k of network $\mathcal{N}$, less than $\forall \epsilon$ where $0 < \epsilon < 1$, as k tends towards infinity:

$$\lim_{k \rightarrow \infty} D_k < \epsilon \quad (19)$$

Proof. To prove Decentralization, it should define and analyze the variance of the number of responses X given by both supervisor nodes SN and validator nodes N over a large number of rounds k . The number of nodes SN and N is denote as n .

Let us consider X^{SN} and X^N as the number of responses from supervisors and validator nodes respectively. Denoted the number of responses from supervisor node SN as X_k^{SN} , and from validator node N as X_k^N , the total number of nodes be n .

According to (15), $SN \setminus C = \{N_1 \dots N_n\} = N$, while $|SN| = n$, it expect $\{SN_1 \setminus C \cup SN_2 \setminus C \cup \dots \cup SN_i \setminus C\} = \{N_1 \cup N_2 \cup \dots \cup N_i\}$, therefore it get (20):

$$X_k^{SN} = X_k^N \quad (20)$$

As long as $SN = N$, each node regardless of its type, handles the equivalence amount of requests over $k \rightarrow \infty$ rounds. Since (20) and $\mathcal{N} = \{SN, N\}$, this work use X_k^i instead of $X_k^{N_i}$.

For each round, the probability of node i to handle request be $p(i) = 1/n$. According to *Law of large numbers*, when $k \rightarrow \infty$, the actual number of responses X_k^i for each node will converge to its expected value, where $X_k^i = k/n \pm \epsilon$. According to the definition of average number $\bar{X}$, it can be written as follow (21):

$$\bar{X} = \frac{\sum_{i=1}^n X_k^i}{n} \quad (21)$$

As the $X_k^i = k/n \pm \epsilon$, according to (21), it get $\bar{X} = k/n \pm \epsilon$, therefore $D_k = \frac{\sum_{i=1}^n (\epsilon)^2}{n-1}$, this work prove the $\lim_{k \rightarrow \infty} D_k < \epsilon$. According to 4, the decentralization of zkRPC is assured. $\square$

This proof demonstrates that despite the initial concerns regarding the presence of supervisor nodes, zkRPC's design and operational mechanics ensure a decentralized and robust network where no single node or group of nodes centralizes control.

4.4. Fault tolerance

Due to zkRPC operating on a decentralized network, the ability to maintain system liveness is a crucial metric. As zkRPC relies on supervisor nodes for the distribution of Zero-Knowledge Proof (ZKP) evidence, its fault tolerance differs from that of traditional P2P networks. This differentiation is described as follows:

Lemma 5 (Fault Tolerance of zkRPC). Denoted the zkRPC network as $\mathcal{N} = \{SN, N\}$. If there exists one supervisor node $SN_i \models \mathcal{A}$, where $SN_i \in SN$, $N_j \in N$. The network $\mathcal{N} \models \mathcal{A}$.

Proof. To prove fault tolerance, the zkRPC shall stay alive when there exists at least one alive SN . From the definition of $\mathcal{N}$, it can be derived that when $SN \models \mathcal{A} \wedge N \models \mathcal{A} \rightarrow \mathcal{N} \models \mathcal{A}$. Therefore, Lemma 5 can be expressed as (22):

$$SN_i \models \mathcal{A} \wedge N_j \models \mathcal{A} \rightarrow \mathcal{N} \models \mathcal{A} \quad (22)$$

Firstly, this work simplify the expression by the expression (15), where $SN_i = \{C, N_1, \dots, N_n\}$ shown any supervisor must have contained ordinary nodes, as the SN itself can serve as N . Therefore, (22) can be converted to (23)

$$SN_i \models \mathcal{A} \rightarrow \mathcal{N} \models \mathcal{A} \quad (23)$$

As network $\mathcal{N} = \{SN, N\}$, when $|SN| = 1$, $SN_i \in SN$, it get $SN_i \models \mathcal{A} \rightarrow SN \models \mathcal{A}$. According to (15), when $|SN| = 1$, $N \in SN$, it get $SN_i \models \mathcal{A} \rightarrow N \models \mathcal{A}$.

Table 2
Parameter used in the network.

Category	Parameter	Value
Network	LATENCY	0.12 s
Network	BANDWIDTH	10 Mb/s
Network	NODES	100
ZKP	INSTANCES	115
ZKP	ENV	Circom (WASM)
Bitcoin	BLOCK_SIZE	1 Mb
Bitcoin	HEIGHT	100
Bitcoin	BLOCK_TIME	600 s

According to [Definition 3](#), as long as SN, N is alive, $\mathcal{N} \models \mathcal{A}$. According to [\(23\)](#), as long as there exists at least one supervisor, the zkRPC shall be alive. $\square$

Overall, zkRPC's fault tolerance requirements are slightly higher than those of traditional P2P networks, as it necessitates at least one supervisor node to remain operational. While faults in ordinary nodes might affect zkRPC's network, they will not cause the whole network to fail. Additionally, the hardware requirements for supervisor nodes are only comparable to those of a full Bitcoin node, meaning the number of supervisor nodes is not limited by hardware specifications.

5. Evaluation

This section outlines the experimental environment and parameters used to assess the performance of zkRPC. The experiments are designed to evaluate the efficiency of zero-knowledge proofs (ZKP) in zkRPC and the overall communication efficiency within the network.

5.1. Environment setting

This work use the server with 8 processor AMD Ryzen 9 5900HX, 64 GB RAM to deploy and run zkRPC in the local network, where the supervisor nodes are connected Bitcoin Regtest network. Regtest is a special mode for Bitcoin nodes, which allows the daemon of Bitcoin to monitor a local blockchain. This work use scripts written in NodeJs to control the whole Regtest network to simulate the behavior Bitcoin full node. The configuration used in our simulation is shown in [Table 2](#):

The parameter of ZKP needs to be specified. *INSTANCE* can reflect the complexity of the ZKP circuit, this work employ a medium-sized circuit in our case. *ENV* is the compiler used to build R1CS (Statement) of the circuit. Overall, our experimental setup is structured closely to common network conditions to rigorously evaluate the performance of zkRPC [\[26\]](#).

5.2. Efficiency examination

Although mainstream ZKP algorithms can provide compact proofs P , the process of constructing P is limited in efficiency. Specifically, as the complexity of the statement S increases, the speed of constructing proofs also decreases. Practically, traversing the entire chain for each ZKP construction would consume a significant amount of time. Therefore, this paper employs a checkpoint mechanism, where for each block b that needs to be verified, only the blocks up to the previous checkpoint are verified and the corresponding P is provided. The length of the checkpoint directly determines the time required to construct the ZKP.

To demonstrate the efficiency, this work measure the time required to generate and verify ZKP for different lengths of checkpoints under the Circom framework. The experimental results are summarized in the [Table 3](#):

The term *Wires* refers to the complexity of the circuit, where a higher number of wires indicates that more computation is required to construct its Statement. From the table, it is evident that as the length of the checkpoint increases, the size of the Wire also increases. At the

Table 3
Time required to generate and verify ZKP.

Length	Wires	Prover Time (s)	Verifier Time (s)
10	985,433	47.22	0.52
20	1,970,863	102.99	0.61
30	2,956,296	168.41	0.66
40	3,941,729	233.43	0.67

Table 4
Size of ZKP Related data.

Length	Wires	Proof (byte)	Public (byte)	VKey (byte)
10	985,433	806	176	2,641
20	1,970,863	807	176	2,655
30	2,956,296	806	176	2,651
40	3,941,729	808	176	2,677

same time, the overhead of proof also increases linearly, indicating that more time is needed to initialize the network. Fortunately, the final verification process is relatively simple, requiring only a few hundred milliseconds to complete. The final results prove that zkRPC can satisfy the practical scenarios.

5.3. Storage verification

Next, this work consider the storage space, which is one of the parameters of greatest concern in this paper. Similarly, in practice, storage space is still directly related to the range of checkpoints. Specifically, nodes only need to save the parameters required for the ZKP verification process. In $\mathcal{ZV}$ process, the verification key *VKey* can replace the Statement S to reduce the storage space required by validator nodes. Therefore, this paper will examine the sizes of *Public*, *Proof*, and *VKey*, and the results are shown in the [Table 4](#):

From the table, it can obtain the performance of the storage. Overall, the size occupied by the ZKP parameters is almost constant, and their volume is below 4 kb. Compared to the blocks they save, this small space consumption greatly compresses the storage needed for conventional data availability solutions and does not cause a performance bottleneck.

5.4. Communication performance

The communication efficiency of zkRPC is evaluated by measuring the time and resources required to transmit data and proofs between nodes. Given the P2P network architecture employed by zkRPC, the topology of data propagation exhibits inherent randomness. In experiments, this work assumed that blocks are uniformly distributed across validator nodes. This work examined the system's performance under two distinct request scenarios: (1) users randomly request any non-latest block, and (2) users invariably request the most recent block. The outcomes of these scenarios are summarized in [Fig. 11](#):

From the figures, a significant discrepancy in latency between requests for non-latest blocks and the latest block is evident. This disparity is primarily due to the communication with the Bitcoin network and the time required to generate proofs. Details of these observations are as follows:

- **Non-Latest Block Requests:** These requests typically response within 1 s, demonstrating robust performance. Notably, the response time for non-latest blocks is independent of the checkpoint length, indicating efficient data retrieval mechanisms that are not constrained by the length of historical data.
- **Latest Block Requests:** Requests for the latest block require waiting for the supervisor node to retrieve the most recent block from the Bitcoin network, which generally takes between 2 to 30 s. Additionally, as the checkpoint length increases, more time is required to construct proofs, consequently leading to greater overall latency.

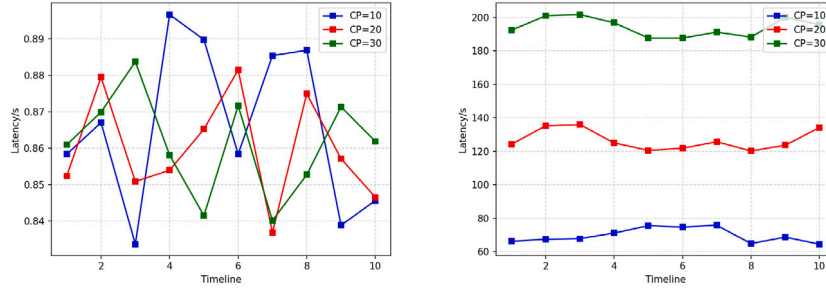


Fig. 11. The latency of zkRPC. (Left) The latency of non-latest block requests. (Right) The latency of the latest block request. Timeline indicate the number of request, there is certain period of time between each request, for non-latest block is 5s, for latest block is 600 s.

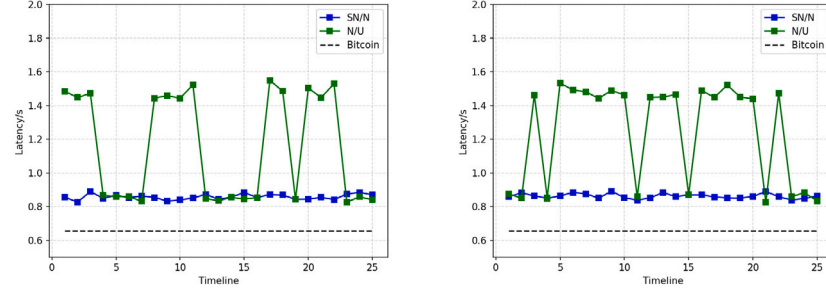


Fig. 12. The latency of different communication pair. *SN/N* is supervisor/validator, *N/U* is validator/user, *Bitcoin* latency serve as baseline. The inter period of request is 5s. (left) the checkpoint length is set as 10. (right) the checkpoint length is set as 30. Clearly, the checkpoint length does not affect the communication latency.

Table 5

Comparison of DA Solution.

Name	Storage(User)	Storage(Node)	Trust Assumption	Update Latency (s)
zkRPC	None	Partial State	Trustless	60
SPV	Block Header(80MB)	Full State	Trustless	P2P latency (≈ 15)
RPC	None	Full State	Trusted	Instant (≤ 5)
DAC	None	Full State	Trusted	Instant (≤ 5)
DAS	Block Header(80MB) + Sample	Full State	Trustless	P2P latency (≈ 15)

Despite these delays, considering the typical Bitcoin network block time of approximately 10 min, even with extensive checkpoint lengths, the system can still meet the practical application scenarios. Moreover, while the initial transmission of proofs may consume more time, once the proofs are constructed, subsequent operations revert to normal latency.

In addition, it is necessary to testify the communication cost between supervisor/validator, and validator/user. According to the zkRPC workflow, their latency should be different, and the result is shown in Fig. 12:

From the figure, the latency between supervisor/validator is relatively stable, mainly due to all of the supervisor carry the full blockchain data. Meanwhile, the latency between validator/user has a notable pattern. It is because the validator may not store the required data, therefore it communicate with other peer validator, and bring an extra round of communication, and increase the latency.

5.5. Comparison

To demonstrate the performance of zkRPC further, this work compare zkRPC with other data availability solutions. Due to the flexible nature of most DA approaches, this work have primarily used qualitative descriptions to highlight the differences between various solutions, as illustrated in Table 5.

In the table, *Update Latency* refers to the delay in receiving the latest state after updates occur in the blockchain. This work derived these average values from existing works [27,28]. *Trust Assumption* indicates whether the solution requires users to trust the corresponding nodes.

Trustless implies that users can verify the data sent without needing to trust the sender, while *Trusted* means that users cannot independently verify the data's correctness, thereby requiring the node's honesty.

From the table, zkRPC shows a significant advantage in storage, particularly for DA users who can quickly verify data sent by nodes without storing any information themselves. Additionally, in networks with a certain number of nodes, zkRPC significantly reduces storage requirements for validator nodes. However, as a trade-off, the computation involved in ZKPs introduces additional overhead, resulting in notably higher update latency compared to other solutions. Yet, considering that the Bitcoin network updates approximately every 10 min, this overhead is within a reasonable range.

These evaluations confirm that zkRPC's design, while exhibiting the latency in processing the latest block requests, remains practical and efficient for real-world applications, particularly given the inherent latency within the underlying Bitcoin network.

6. Discussion

To comprehensively introduce the zkRPC, this work discussed several potential drawback that may affect the overall performance, and other special concerns.

Q1: Latency: The proving time for ZKP is considerably long and significantly impacts the request latency. Is there an effective solution to this issue?

In practice, this work employ a trick to mitigate the delay caused by proof construction and prevent network congestion. When a node

Table 6
Throughput Comparison of Transaction.

Name	Bitcoin Client (Local)	Blockstream	zkRPC (Local)
Size (byte)	1154	1176	4844
Throughput (tx/s)	221.2	2.2	207.2

receives the latest block, instead of waiting for the proof to be constructed, the supervisor node temporarily hands over the latest block along with the sub-latest block and its proof, denoted as $\langle b_n, b_{n-1}, P_{n-1} \rangle$, to the validator nodes and users. Possessing b_{n-1} allows users to easily verify $V(b_n)$. Although this method requires the transmission of an additional 1MB of data, it is considerably more efficient than waiting for proof generation. However, ZKPs does bring shortcoming to this work, increase the network latency. It can be further improved by employ more efficient ZKPs schema.

Additionally, ZK hardware acceleration can further enhance computational efficiency and reduce verification time [29]. However, such hardware may introduce new security risks. By employing fault detection techniques, hardware errors and errors arising from cryptographic calculations like *SHA3-256*, can be minimized as much as possible, thereby improving the overall reliability of ZKP while achieving efficiency gains in consensus [30,31]. These considerations will be a primary focus to further improve this work.

Q2: Redundancy: There are multiple supervisors in zkRPC; when a single validator belongs to several supervisors, it may lead to the redundancy of the network.

While it may seem inefficient, a certain degree of redundancy is beneficial for the overall robustness of a decentralized network to prevent nodes failure. Moreover, due to the nature of block requests — where the latest blocks are frequently accessed, and historical blocks are nearly neglected — it is practical for any validator to at least store the most recent 10 blocks.

Q3: Centralized: Supervisors appear to be a centralized concept, while validator nodes depend on them for initialization and updates.

Indeed, supervisors play a significant role in zkRPC, but this does not imply they are irreplaceable. First, almost all availability requests from users are completed by validator nodes, with supervisors having no interference in this process. Additionally, supervisors do not have stringent operational requirements compared to nodes in many PoS networks; any server capable of running a Bitcoin full node suffices. This is because supervisors have no opportunity for malfeasance as they must provide verifiable ZKP to be trusted by other nodes, making the process entirely trustless.

Q4: Throughput: For a decentralized system, the overall throughput should be an important metric.

The throughput of zkRPC was not specifically addressed in the paper primarily because zkRPC does not participate in the consensus process of the Bitcoin Network. As such, zkRPC functions more like a traditional distributed network rather than a blockchain network. Consequently, its throughput is largely determined by the network speed of its nodes and the effectiveness of optimizations related to memory, message handling, network router/region, and CDN configurations. Focusing on maximizing throughput in these terms diverts from the themes of our discussion, which are centered around the trustless characteristics of zkRPC. Nonetheless, this work have testified the overall throughput of the transaction request without special optimization, and the result is shown in Table 6:

From the table, it can find out that, zkRPC performances are close to ordinary *get_transaction* as the package of transaction request of zkRPC is about 5 kb, while the transaction data of Bitcoin RPC is about 1 kb. However, the *Blockstream API* has an extremely low throughput, while its size is barely equal to the local Bitcoin client. The result indicated the network condition and the node settings have a vital influence on

the final throughput, which is out of the topic of this paper.

Q5: Layer: What is the difference between zkRPC to a typical data availability layer.

Data availability layers are specialized, modular facilities designed for blockchain networks, and zkRPC is tailored for the Bitcoin network. Typical data availability layers like *Celestia*, *Avail*, and *EigenDA* cater to multiple blockchain networks by hosting, accessing, and verifying them using a combination of on-chain and off-chain data storage methods. These layers are oriented towards nodes across various blockchain networks rather than targeting ordinary blockchain users. In contrast to these data availability layers, zkRPC functions more as a DA service in Bitcoin rather than a modular blockchain network infrastructure.

Q6: Complexity: What about the complexity of zkRPC.

For computational complexity, it is mainly depend on the ZKPs schema and the number of constrained. For schema, zkRPC use *Groth16*, which is relative fast in proof/verify [32]. As for constrain, according to algorithm 1 and 2, the constrained of zkRPC should be *3022440 non-linear constraints*, it is more complicated than ordinary Circom circuit compare to benchmark [33]. Fortunately, the inter-block period of Bitcoin is long enough to tolerate this drawback.

For communication complexity, according to the workflow of supervisor and validator node, In optimistic scenario, the user can directly get response from validator, and only takes one round of communication, render the complexity be $O(n)$. In the pessimistic scenario, the validator should asked peers for valid response, which takes two round of communication and the complexity is $O(n^2)$. Overall, the communication complexity is acceptable even in unfavorable situation.

7. Related work

7.1. Concept of data availability in blockchain

In blockchain technology, the concept of data availability arose from the need for scalability. Researchers have observed that requiring all nodes in a blockchain to maintain the entire chain data leads to excessive redundancy and reduces network efficiency [34]. Thus, data availability has become a broader concept that any mechanism allowing nodes to partition their state can be considered a data availability solution. Notable examples include sharding networks [35] and Layer-2 Networks [36], which establish separate sub-networks that store only a portion of the data.

7.2. Data availability solutions

To enable nodes to forego storing the complete state, typical solutions include distributed data availability based on erasure coding and centralized data availability solutions based on Data Availability Committees (DAC). Representative works include Lazyledger [37] and Layer-2 DAC [9], specifically:

Lazyledger explores the potential of using erasure codes to reduce blockchain storage requirements. Erasure coding allows network nodes to store only parts of the data while being able to reconstruct the complete data state when necessary. This method ensures data integrity and accessibility, even when some nodes are unavailable, by adding a minimal amount of redundancy.

The Layer-2 DAC approach involves forming a dedicated organization to manage and verify the integrity and availability of data. Meanwhile, it establishes a token-based incentive and penalty system to ensure the DAC operates honestly. While this method offers high efficiency and performance, it generally relies on some form of centralized management, introducing a strong trust assumption.

7.3. Emerging works in data availability

As the concept of data availability has evolved, several drawbacks have emerged due to the pursuit of efficiency. Consequently, a range of efforts has begun to develop improvements to address potential centralization and security issues in existing solutions. Recent works include proof-of-storage [38] and modular indexer [39], specifically:

The Modular Indexer introduces a fully user-verified execution layer for Bitcoin, enabling Turing-complete operations off-chain. This innovative approach overcomes Bitcoin's scripting limitations, facilitating complex applications and protocols. By ensuring that all operations are verified by users, the system upholds the core principles of decentralization and trustlessness in cryptocurrency. This method allows for more flexible and varied implementations on the Bitcoin network without compromising security or integrity.

Another work proof-of-storage focuses on enhancing Layer-2 blockchain networks to improve scalability and data availability without compromising decentralization. The new techniques involve optimized handling of off-chain data, reducing the main chain's load while ensuring robust security measures. These improvements address inherent limitations in previous roll-up solutions, proposing methods that maintain data integrity and minimize risks associated with data unavailability, thereby paving the way for more efficient and secure blockchain applications.

7.4. Zero-knowledge proof and blockchain

The ZKP is designed to keep the private information in secret. This attribute is perfectly fit the needs of blockchain privacy. Currently, ZKPs is wildly used in the *rollup*, *identity* and *anonymity*. For rollup, ZKPs provide a way for layer-2 network to align the security features with layer-1 blockchain, by verify a ZKP that prove that all transaction batch in layer-2 network is valid. Notable implementation including Starknet, zkSync [40,41]. For identity, ZKP can be used to prove a user own an authorized ID (like google account) without reveal the personal information, like zkLogin [42]. For anonymity, the blockchain leverage ZKP to cover the detail of a transaction, make it only reveal to the dealer and receiver, like Zerocash [43].

On the security aspect, unfortunately, the mainstream ZKPs is unable to resist quantum computing attacks. However, it is theoretically possible to apply such design in the current ZKPs schema with certain modification in elliptic curves [44]. There are plenty amount of existing work that focus on solving this issue on ZKPs and blockchain [45].

8. Conclusion

zkRPC effectively addresses the challenges of data availability in Bitcoin networks by employing zero-knowledge proofs and peer-to-peer architecture. The zkRPC demonstrates significant improvements in data verification and storage while ensuring security and decentralization. Our approach confirms zkRPC's capability to facilitate efficient and reliable Bitcoin data availability operations. The user can fetch data from Bitcoin without any local storage, and be able to verify any data by themselves.

While zkRPC shows promising results, future enhancements are needed to optimize proof generation times and reduce network congestion, which include:

- **Enhanced ZKPs Schema:** The current implementation of ZKPs has not yet reach the best performance. More efficient schema shall be deployed.
- **Scalability:** The throughput and network performance of zkRPC can be further improve, meanwhile, its compatibility to other blockchain is also regard valuable.

Additionally, exploring more dynamic checkpoint strategies could improve the system's scalability and responsiveness. These improvements will refine zkRPC's performance and expand its practical applications.

CRedit authorship contribution statement

Kundu Chen: Writing – original draft, Visualization, Software, Methodology, Formal analysis, Conceptualization. **Jie Luo:** Writing – review & editing, Validation, Supervision, Investigation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Kundu Chen reports a relationship with Bitslab that includes: consulting or advisory. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the State Key Laboratory of Complex & Critical Software Environment (SKLCCSE).

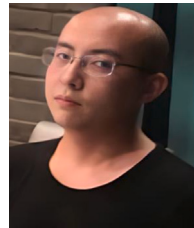
Data availability

No data was used for the research described in the article.

References

- [1] statista, Size of the bitcoin blockchain from january 2009 to september 23, 2024, 2024, <https://www.statista.com/statistics/647523/worldwide-bitcoin-blockchain-size/>.
- [2] P.K. Kaushal, A. Bagga, R. Sobti, Evolution of bitcoin and security risk in bitcoin wallets, in: 2017 International Conference on Computer, Communications and Electronics (Comptelix), IEEE, 2017, pp. 172–177.
- [3] N. Li, M. Qi, Q. Wang, S. Chen, Bitcoin inscriptions: Foundations and beyond, 2024, arXiv preprint [arXiv:2401.17581](https://arxiv.org/abs/2401.17581).
- [4] A. Tripathi, Ordinal theory handbook, 2024.
- [5] K. Gogol, J. Messias, M.I. Silva, B. Livshits, The writing is on the wall: Analyzing the boom of inscriptions and its impact on rollup performance and cost efficiency, 2024, arXiv preprint [arXiv:2404.11189](https://arxiv.org/abs/2404.11189).
- [6] X. Fan, B. Niu, Z. Liu, Scalable blockchain storage systems: research progress and models, *Computing* 104 (6) (2022) 1497–1524.
- [7] H. Brakmić, H. Brakmić, Bitcoin practice, in: Bitcoin and Lightning Network on Raspberry Pi: Running Nodes on Pi3, Pi4 and Pi Zero, Springer, 2019, pp. 225–261.
- [8] K. Li, J. Chen, X. Liu, Y.R. Tang, X. Wang, X. Luo, As strong as its weakest link: How to break blockchain dapps at rpc service., in: NDSS, 2021.
- [9] E.N. Tas, D. Boneh, Cryptoeconomic security for data availability committees, in: International Conference on Financial Cryptography and Data Security, Springer, 2023, pp. 310–326.
- [10] P. Sheng, B. Xue, S. Kannan, P. Viswanath, ACeD: Scalable data availability oracle, in: Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part II 25, Springer, 2021, pp. 299–318.
- [11] M. Hall-Andersen, M. Simkin, B. Wagner, Foundations of data availability sampling, *Cryptol. ePrint Arch.* (2023).
- [12] M. Yu, S. Sahraei, S. Li, S. Avestimehr, S. Kannan, P. Viswanath, Coded merkle tree: Solving data availability attacks in blockchains, in: International Conference on Financial Cryptography and Data Security, Springer, 2020, pp. 114–134.
- [13] J. Hasan, Overview and applications of zero knowledge proof (ZKP), *Int. J. Comput. Sci. Netw.* 8 (5) (2019) 2277–5420.
- [14] X. Sun, F.R. Yu, P. Zhang, Z. Sun, W. Xie, X. Peng, A survey on zero-knowledge proof in blockchain, *IEEE Netw.* 35 (4) (2021) 198–205.
- [15] A. Gervais, G.O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, S. Capkun, On the security and performance of proof of work blockchains, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, 2016, pp. 3–16.
- [16] B. Cao, Z. Zhang, D. Feng, S. Zhang, L. Zhang, M. Peng, Y. Li, Performance analysis and comparison of pow, PoS and DAG based blockchains, *Digit. Commun. Netw.* 6 (4) (2020) 480–485.
- [17] E.N. Tas, D. Tse, F. Gai, S. Kannan, M.A. Maddah-Ali, F. Yu, Bitcoin-enhanced proof-of-stake security: Possibilities and impossibilities, in: 2023 IEEE Symposium on Security and Privacy, SP, IEEE, 2023, pp. 126–145.
- [18] Y. Gong, Y. Jin, Y. Li, Z. Liu, Z. Zhu, Analysis and comparison of the main zero-knowledge proof scheme, in: 2022 International Conference on Big Data, Information and Computer Network, BDICN, IEEE, 2022, pp. 366–372.

- [19] K. Bagheri, Z. Pindado, C. Ràfols, Simulation extractable versions of groth's zk-SNARK revisited, in: *International Conference on Cryptology and Network Security*, Springer, 2020, pp. 453–461.
- [20] M. Bellés-Muñoz, M. Isabel, J.L. Muñoz-Tapia, A. Rubio, J. Baylina, Circom: A circuit description language for building zero-knowledge applications, *IEEE Trans. Dependable Secure Comput.* (2022).
- [21] J.A. Garay, A. Kiayias, N. Leonardos, The bitcoin backbone protocol: Analysis and applications, *J. ACM* (2015).
- [22] B. Koziel, A. Jalali, R. Azarderakhsh, D. Jao, M. Mozaffari-Kermani, NEON-SIDH: Efficient implementation of supersingular isogeny diffie-hellman key exchange protocol on ARM, in: *Cryptology and Network Security: 15th International Conference, CANS 2016, Milan, Italy, November 14–16, 2016, Proceedings 15*, Springer, 2016, pp. 88–103.
- [23] M. Bisheh-Niasar, R. Azarderakhsh, M. Mozaffari-Kermani, Cryptographic accelerators for digital signature based on ed25519, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 29 (7) (2021) 1297–1305.
- [24] A. Tripathi, Calculating merkle root and merkle proof in bitcoin, 2020, <https://ashutosh-tripathi.medium.com/calculating-merkle-root-and-proof-of-inclusion-in-bitcoin-ebd65f504af9>.
- [25] C. Decker, J. Seidel, R. Wattenhofer, Bitcoin meets strong consistency, in: *Proceedings of the 17th International Conference on Distributed Computing and Networking*, 2016, pp. 1–10.
- [26] F. Zola, C. Pérez-Solà, J.E. Zubia, M. Eguimendia, J. Herrera-Joancomartí, Kriptosare. gen, a dockerized bitcoin testbed: analysis of server performance, in: *2019 10th IFIP International Conference on New Technologies, Mobility and Security, NTMS*, IEEE, 2019, pp. 1–5.
- [27] L. Wan, D. Eysers, H. Zhang, Evaluating the impact of network latency on the safety of blockchain transactions, in: *2019 IEEE International Conference on Blockchain (Blockchain)*, IEEE, 2019, pp. 194–201.
- [28] C. Decker, R. Wattenhofer, Information propagation in the bitcoin network, in: *IEEE P2P 2013 Proceedings*, IEEE, 2013, pp. 1–10.
- [29] H. Zhao, D. Ding, F. Wang, P. Hua, N. Wang, Q. Wu, Z. Chai, Hardware acceleration of number theoretic transform for zk-SNARK, *Eng. Rep.* (2022) e12639.
- [30] M.M. Kermani, A. Jalali, R. Azarderakhsh, Lightweight error detection architectures through swapping the shares for a subset of S-boxes, in: *2018 IEEE 61st International Midwest Symposium on Circuits and Systems, MWSCAS*, IEEE, 2018, pp. 578–581.
- [31] S. Bayat-Sarmadi, M. Mozaffari-Kermani, A. Reyhani-Masoleh, Efficient and concurrent reliable realization of the secure cryptographic SHA-3 algorithm, *IEEE Trans. Computer-Aided Design Integr. Circuits Syst.* 33 (7) (2014) 1105–1109.
- [32] J. Groth, On the size of pairing-based non-interactive arguments, in: *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Vienna, Austria, May 8–12, 2016, Proceedings, Part II 35, Springer, 2016, pp. 305–326.
- [33] C. Steidtmann, S. Gollapudi, Benchmarking ZK-circuits in circom, *Cryptol. ePrint Arch.* (2023).
- [34] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, A. Gervais, Sok: Layer-two blockchain protocols, in: *Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers 24*, Springer, 2020, pp. 201–226.
- [35] F. Hashim, K. Shuaib, N. Zaki, Sharding for scalable blockchain networks, *SN Comput. Sci.* 4 (1) (2022) 2.
- [36] A. Gangwal, H.R. Gangavalli, A. Thirupathi, A survey of layer-two blockchain protocols, *J. Netw. Comput. Appl.* 209 (2023) 103539.
- [37] M. Al-Bassam, Lazyledger: A distributed data availability ledger with client-side smart contracts, 2019, arXiv preprint [arXiv:1905.09274](https://arxiv.org/abs/1905.09274).
- [38] C. Huang, R. Song, S. Gao, Y. Guo, B. Xiao, Data availability and decentralization: New techniques for zk-rollups in layer 2 blockchain networks, 2024, arXiv preprint [arXiv:2403.10828](https://arxiv.org/abs/2403.10828).
- [39] H. Wen, H. Liu, S. Tang, S. Cao, Y. Feng, et al., Modular indexer: Fully user-verified execution layer for meta-protocols on bitcoin, *Cryptol. ePrint Arch.* (2024).
- [40] J. Gorznyi, M. Derka, A rollup comparison framework, 2024, arXiv preprint [arXiv:2404.16150](https://arxiv.org/abs/2404.16150).
- [41] S. Chaliasos, I. Reif, A. Torralba-Agell, J. Ernstberger, A. Kattis, B. Livshits, Analyzing and benchmarking ZK-rollups, *Cryptol. ePrint Arch.* (2024).
- [42] F. Baldimtsi, K.K. Chalkias, Y. Ji, J. Lindstrøm, D. Maram, B. Riva, A. Roy, M. Sedaghat, J. Wang, Zklogin: Privacy-preserving blockchain authentication with existing credentials, 2024, arXiv preprint [arXiv:2401.11735](https://arxiv.org/abs/2401.11735).
- [43] K. Naganuma, M. Yoshino, H. Sato, T. Suzuki, Auditable zerocoin, in: *2017 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, IEEE, 2017, pp. 59–63.
- [44] B. Koziel, R. Azarderakhsh, M.M. Kermani, D. Jao, Post-quantum cryptography on FPGA based on isogenies on elliptic curves, *IEEE Trans. Circuits Syst. I. Regul. Pap.* 64 (1) (2016) 86–99.
- [45] T.M. Fernandez-Carames, P. Fraga-Lamas, Towards post-quantum blockchain: A review on blockchain cryptography resistant to quantum computing attacks, *IEEE Access* 8 (2020) 21091–21116.



Kundu Chen, Ph.D., Candidate in Beihang University, supervised by Prof. Wei Li, Prof. Jie Luo since 2016. He have notable works in the industry, such as the formal verification of Aptos framework, Move analyzer, etc. Primary research in the field of blockchain, blockchain consensus, smart contract and applied cryptograph.



Jie Luo (Member, IEEE) received the B.S. degree in mathematics from Peking University, Beijing, China, in 2003, and the Ph.D. degree in computer science from Beihang University, Beijing, in 2012. From 2012 to 2019, he was an Assistant Professor with the School of Computer Science and Engineering, Beihang University, where he has been an Associate Professor since 2019. His research interests include mathematical logic, knowledge acquisition and reasoning, crowd intelligence theory, and formal methods for bug localization.